Dockerized Construction Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	SHIWANI PATHAK	EN22CS301921
2.	SHRISHTI SAHU	EN22CS301936
3.	SHREYAS	EN22CS301934
4.	SANSKAR JAIN	EN22CS301868
5	RAJVEER SINGH CHANGAL	EN23CS3T1009

Group Name: Group 03D08

Project Number: DO-38

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025 - 2026

TABLE OF CONTENTS

1. **Problem Statement & Objectives**
   1.1 Problem Statement
   1.2 Project Objectives
   1.3 Scope of the Project
2. **Proposed Solution**
   2.1 Key Features
   2.2 Overall Architecture / Workflow
      2.2.1 Application Workflow
      2.2.2 DevOps Workflow
      2.2.3 Architecture Explanation
   2.3 Tools & Technologies Used
3. **Results & Output**
   3.1 Screenshots / Outputs
      Figure 1: Frontend Application Interface
      Figure 2: Kubernetes Pods Status
      Figure 3: Kubernetes Services
      Figure 4: Docker Images
      Figure 5: CI/CD Pipeline Execution
      Figure 6: Application Deployment on AWS EC2
   3.2 Reports / Models
      Figure 7: Generated Construction Schedule
      Figure 8: Risk & Severity Analysis
      Figure 9: Gantt Chart Visualization
   3.3 Key Outcomes
4. **Conclusion**
5. **Future Scope & Enhancements**

Problem Statement & Objectives

1. Problem Statement

The objective of this project is to design and deploy a containerized construction planning service using modern DevOps practices. The system aims to automate construction planning, scheduling, and risk analysis while ensuring portability, scalability, and efficient deployment across environments using Docker, Kubernetes, and cloud infrastructure.

2. Project Objectives

The primary objectives of this project are as follows:

- **To containerize the application using Docker with optimized images**
  The application is packaged into lightweight Docker containers using optimized base images such as Python slim and Nginx Alpine. Multi-stage builds are used to reduce image size, improve performance, and enhance security by minimizing unnecessary dependencies.

- **To implement a microservices-based architecture**
  The system is designed by separating the frontend and backend into independent services. This separation ensures modular development, easier maintenance, and the ability to independently scale or update components without affecting the entire system.

- **To deploy the application using Kubernetes for orchestration**
  Kubernetes is used to manage containerized applications through deployments and services. It provides automated container management, load balancing, self-healing capabilities, and efficient resource utilization.

- **To enable CI/CD automation using GitHub Actions**
  A Continuous Integration pipeline is implemented using GitHub Actions to automatically build Docker images whenever code changes are pushed. This ensures faster development cycles, reduces manual errors, and maintains consistency in builds.

- **To host and run the application on AWS EC2 infrastructure**
  The application is deployed on an AWS EC2 instance, providing a cloud-based environment for running Kubernetes. This ensures accessibility over the internet and simulates real-world deployment scenarios.

- **To ensure scalability, portability, and high availability**
  The system is designed to support horizontal scaling through Kubernetes deployments. Containers ensure portability across environments, while

Kubernetes ensures high availability through automatic restarts and load distribution.

- **To implement a complete DevOps workflow**
  The project integrates version control, containerization, orchestration, and automation tools to demonstrate a full DevOps lifecycle from development to deployment.

3. Scope of the Project

The scope of this project includes the design, development, deployment, and automation of a construction planning service using modern DevOps practices.

- **Development of a web-based construction planning application**
  The system provides an interface where users can input construction goals and receive structured outputs such as schedules, risk analysis, and planning steps.
- **Backend API for construction logic and processing**
  The backend, built using Flask, handles task planning, dependency management, scheduling, cost estimation, and risk analysis. It exposes REST APIs for communication with the frontend.
- **Frontend interface for user interaction**
  A user-friendly interface is developed using HTML, CSS, and JavaScript. It allows users to input data and view outputs such as schedules and Gantt charts.
- **Containerization of application components using Docker**
  Both frontend and backend services are containerized separately. Docker ensures consistency across development, testing, and production environments.
- **Orchestration using Kubernetes (K3s cluster)**
  Kubernetes is used to deploy and manage containers using YAML configurations. It handles service discovery, scaling, load balancing, and pod lifecycle management.
- **Deployment on AWS EC2 cloud infrastructure**
  The application is hosted on a cloud-based EC2 instance, enabling external access through public IP and ensuring real-world deployment experience.
- **Implementation of Continuous Integration using GitHub Actions**
  The CI pipeline automatically builds Docker images on every code push, ensuring code integrity and reducing manual deployment efforts.

- **Network configuration and service exposure**
  Kubernetes NodePort services are used to expose the application externally, allowing users to access frontend and backend services through specific ports.

- **System limitations and boundaries**
  The current implementation uses a single-node Kubernetes cluster, which limits scalability compared to multi-node production environments. The system does not include a database or advanced security layers such as authentication.

- **Future extensibility within scope**
  The architecture allows easy integration of additional components such as databases, monitoring tools, auto-scaling mechanisms, and secure communication protocols.

Proposed Solution

1. Key Features

The system incorporates several key features that enhance functionality, performance, and deployment efficiency:

- **Automated Construction Planning**
  The system generates structured construction plans based on user input goals. It uses predefined logic and intelligent processing modules to break down complex construction tasks into manageable steps.
- **Task Scheduling and Dependency Management**
  Tasks are organized in a logical sequence based on dependencies. The system ensures that prerequisite tasks are completed before dependent tasks are executed, resulting in an optimized workflow.
- **Risk Analysis and Severity Detection**
  The system identifies potential delays or issues in construction tasks and provides risk assessments. It categorizes risk levels into severity classes such as low, medium, and high to help users understand project uncertainties.
- **Gantt Chart Visualization**
  A graphical representation of the project timeline is generated in the form of a Gantt chart. This helps users visualize task durations, overlaps, and overall project progress.
- **Docker-Based Containerization**
  The application components are containerized using Docker to ensure consistent execution across environments. Each service (frontend and backend) runs in its own isolated container.
- **Kubernetes-Based Orchestration**
  Kubernetes is used to manage container deployment, scaling, and networking. It ensures high availability, automatic recovery of failed containers, and efficient resource utilization.
- **CI Pipeline Automation**
  Continuous Integration is implemented using GitHub Actions. The pipeline automatically builds Docker images whenever changes are pushed to the repository, ensuring faster and reliable development workflows.
- **Cloud Deployment on AWS**
  The entire system is deployed on an AWS EC2 instance, enabling real-world cloud-based hosting and accessibility through a public IP address.

2. Overall Architecture / Workflow

The system follows a **microservices-based architecture**, where the frontend and backend are separated into independent components and communicate through well-defined interfaces.

Application Workflow

The functional workflow of the application is as follows:

1. The user interacts with the system through a web browser.
2. The frontend interface (served using Nginx) captures the user input.
3. The frontend sends a request to the backend API.
4. The backend processes the request using multiple internal modules such as planner, scheduler, and risk manager.
5. The backend generates outputs including:
    - Task schedule
    - Risk analysis
    - Severity classification
    - Gantt chart
6. The processed response is sent back to the frontend.
7. The frontend displays the results to the user.

Workflow Representation:

User → Browser → Frontend (Nginx) → Backend API (Flask) → Processing Modules → Response → UI

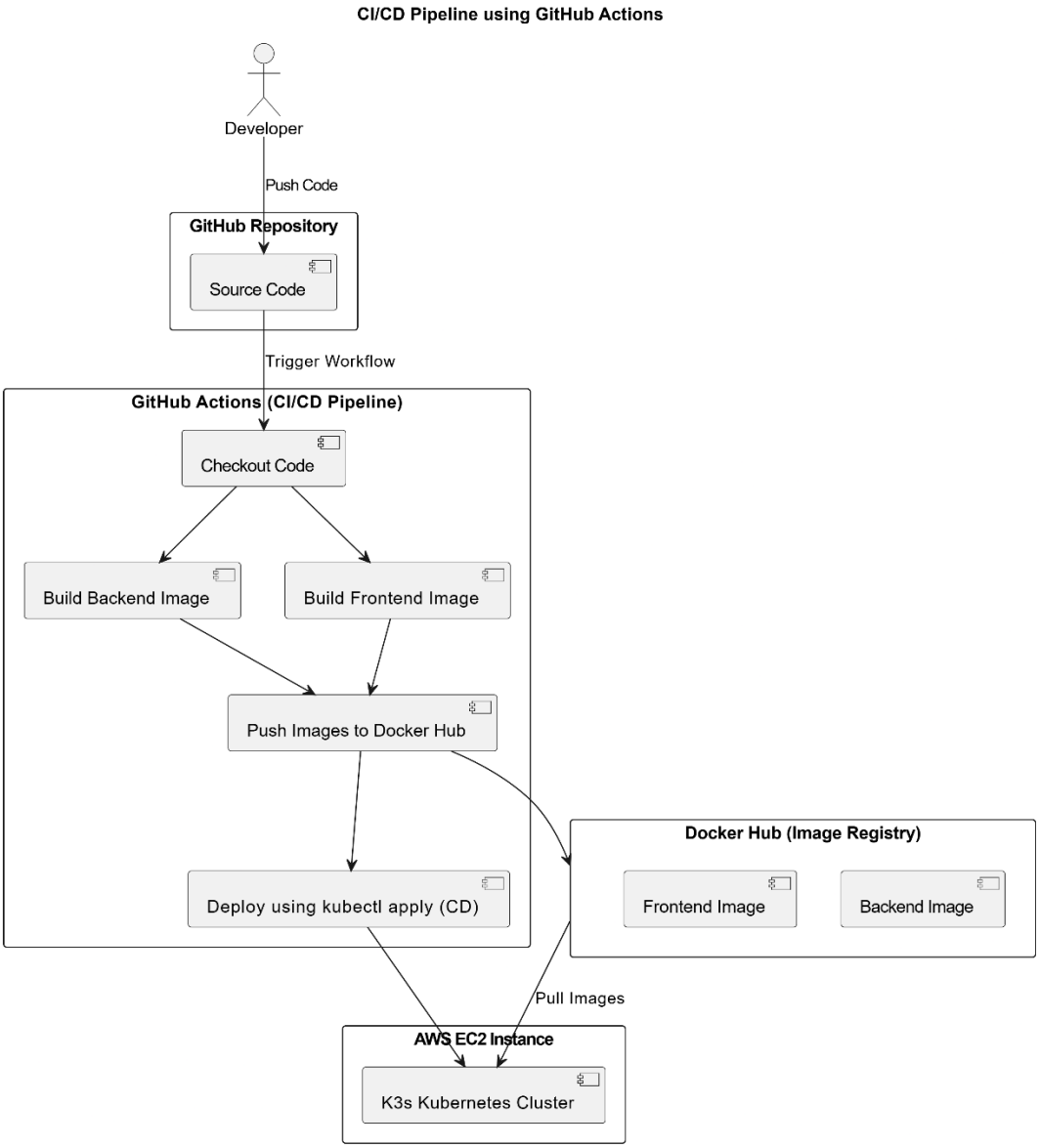
DevOps Workflow

The deployment and automation workflow integrates multiple DevOps tools and follows these steps:

1. The developer writes and updates code locally.
2. Code is pushed to the GitHub repository.
3. GitHub Actions automatically triggers the CI pipeline.
4. The pipeline performs the following:
    - Checks out the code

- Builds Docker images for frontend and backend

5. The application is deployed on Kubernetes running on AWS EC2.
6. Kubernetes manages the containers and ensures continuous availability.



DevOps Workflow Representation:

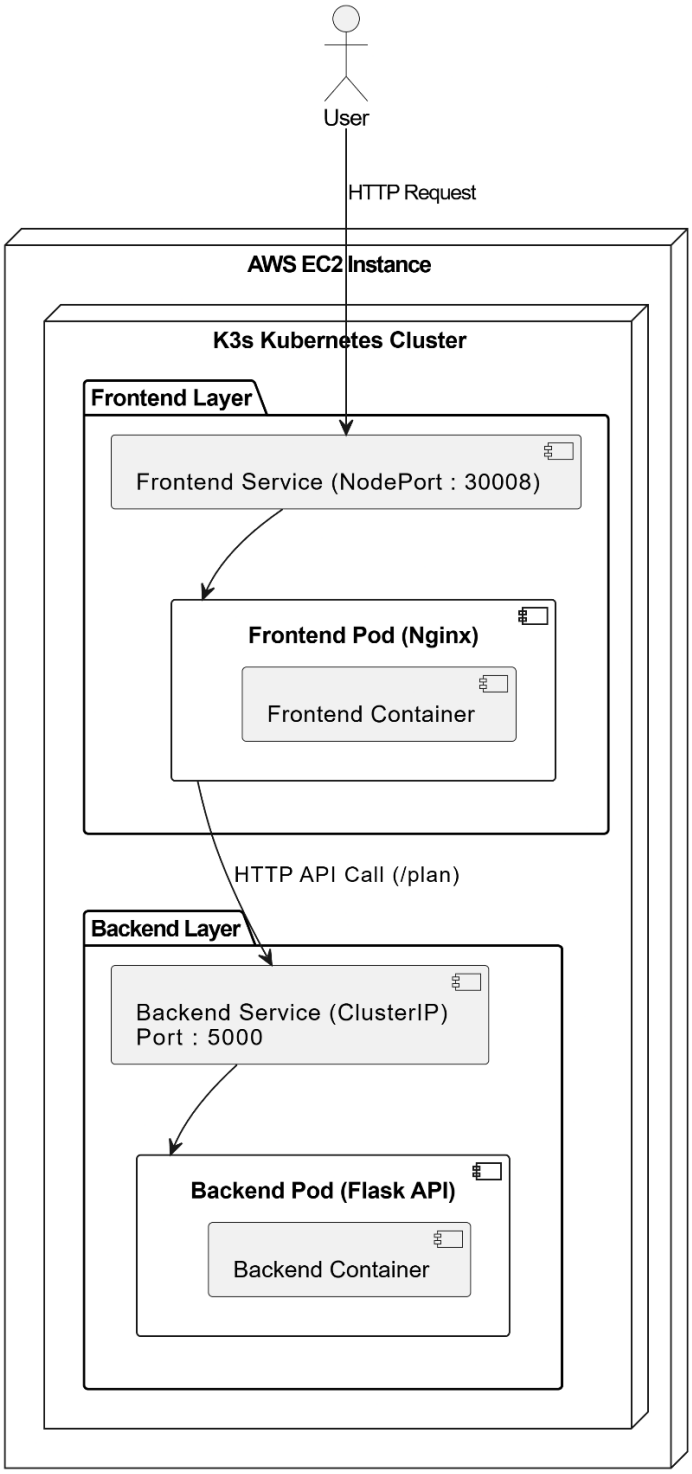
Developer → GitHub → GitHub Actions → Docker Image Build → Kubernetes Deployment → AWS EC2

Architecture Explanation

- The system is deployed on a single-node Kubernetes cluster (K3s) hosted on an AWS EC2 instance.
- The frontend and backend are deployed as separate Kubernetes deployments.

- Kubernetes services are used to expose the application externally via NodePort.
- Internal communication between frontend and backend is handled through Kubernetes service networking.
- Kubernetes ensures self-healing by automatically restarting failed pods.

Kubernetes Architecture for Construction Planning Service



3. Tools & Technologies Used

The project utilizes a combination of development, containerization, orchestration, and cloud technologies to implement a complete DevOps pipeline.

- **Git and GitHub (Version Control System)**
  Used for source code management, version tracking, and collaboration. It also triggers CI workflows when code is pushed.
- **Docker (Containerization Platform)**
  Used to package the application and its dependencies into portable containers. Ensures consistent runtime across environments.
- **Docker Compose (Multi-Container Management)**
  Used during development to run frontend and backend services together in a defined configuration.
- **Kubernetes – K3s (Container Orchestration)**
  A lightweight Kubernetes distribution used to manage deployments, services, scaling, and networking of containers.
- **GitHub Actions (CI/CD Tool)**
  Automates the build process by creating Docker images whenever changes are pushed to the repository.
- **AWS EC2 (Cloud Infrastructure)**
  Provides a virtual machine environment to host the Kubernetes cluster and deploy the application.
- **Python Flask (Backend Framework)**
  Used to build RESTful APIs for processing construction planning logic and handling client requests.
- **HTML, CSS, JavaScript (Frontend Technologies)**
  Used to create an interactive user interface for input and output visualization.
- **Nginx (Web Server)**
  Serves the frontend application and handles HTTP requests efficiently.

Results & Output

1. Screenshots / Outputs

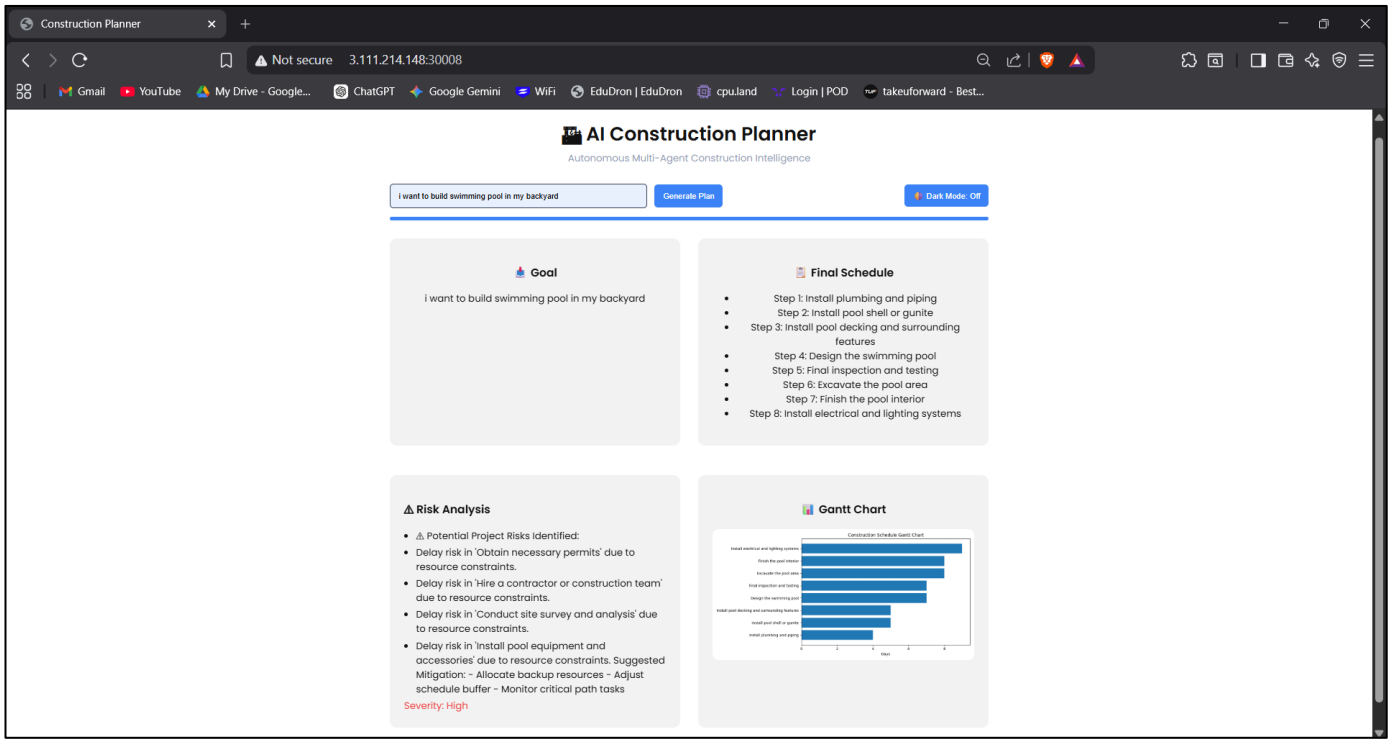


Figure 1: Frontend Application Interface

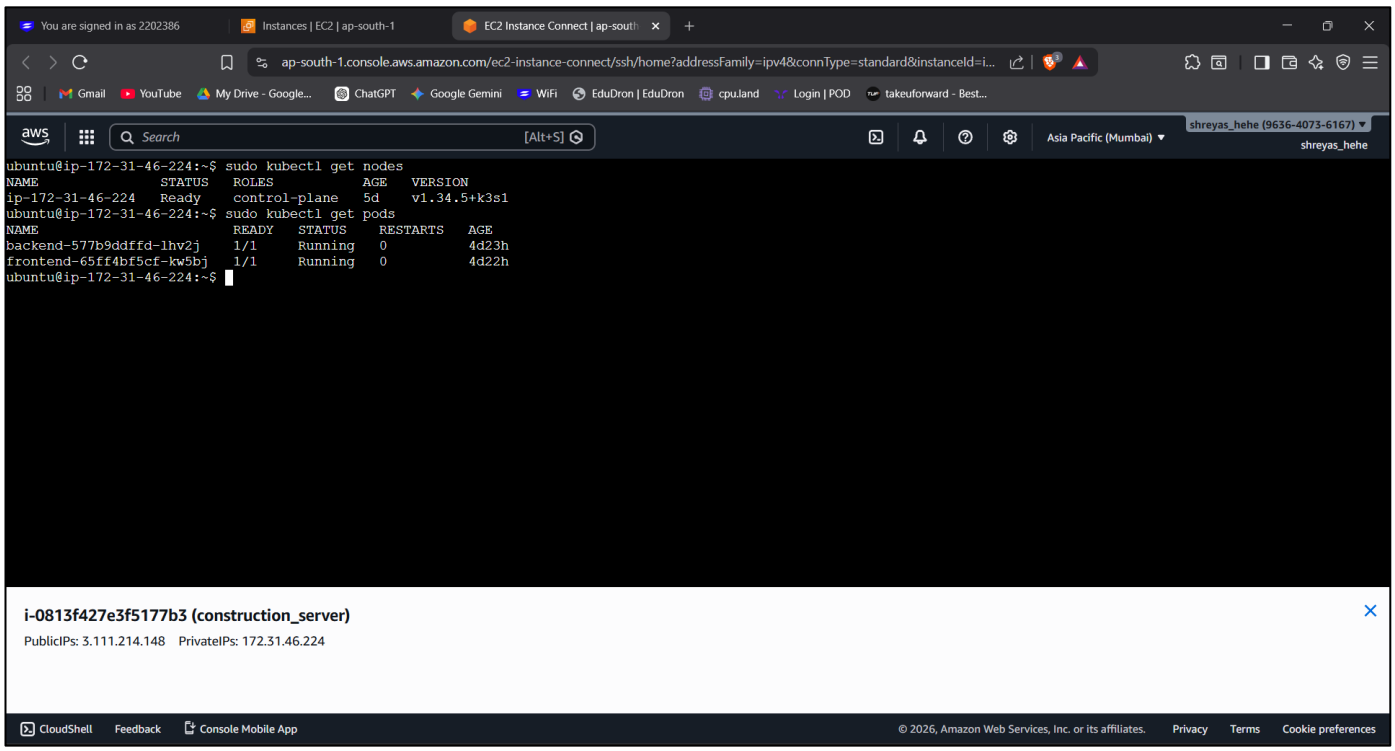


Figure 2: Kubernetes Pods Status

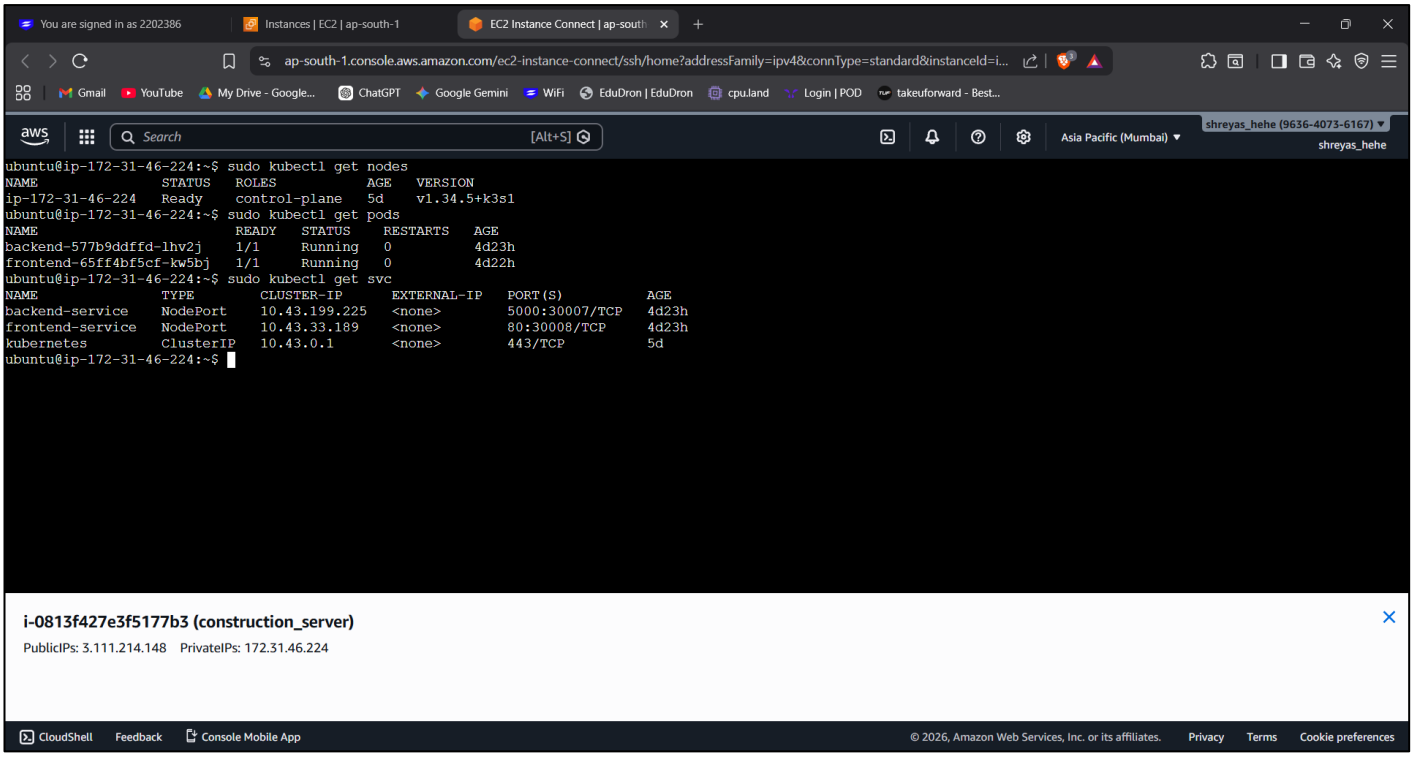


Figure 3: Kubernetes Services

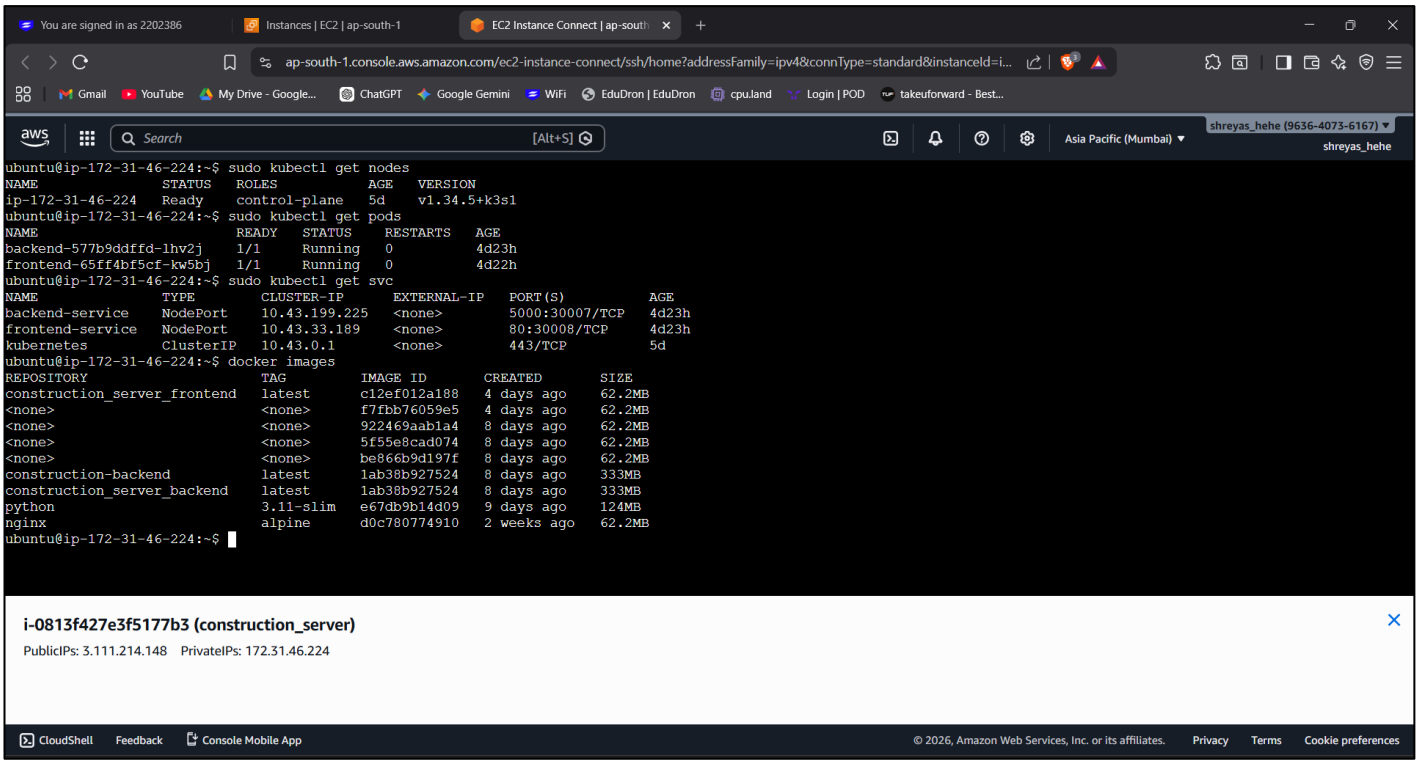


Figure 4: Docker Images

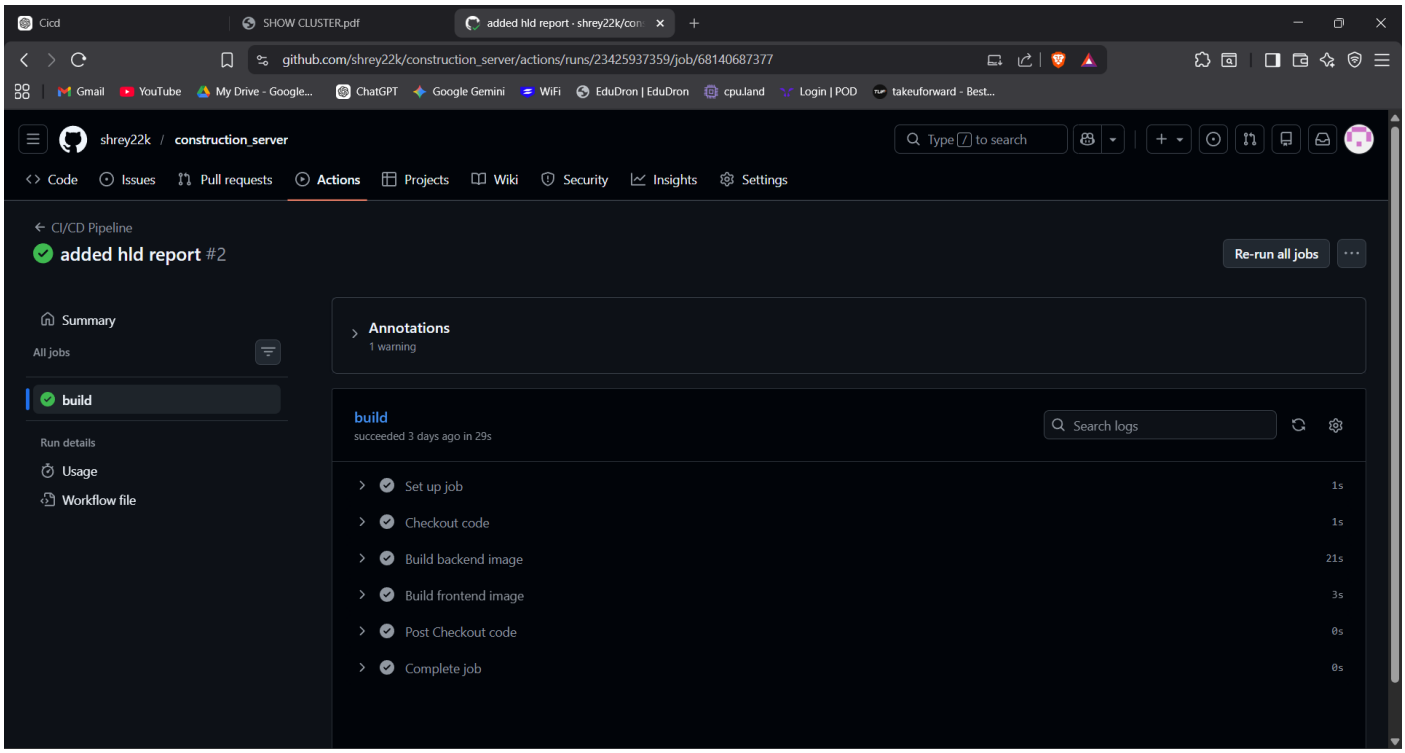


Figure 5: CI/CD Pipeline Execution using GitHub Actions

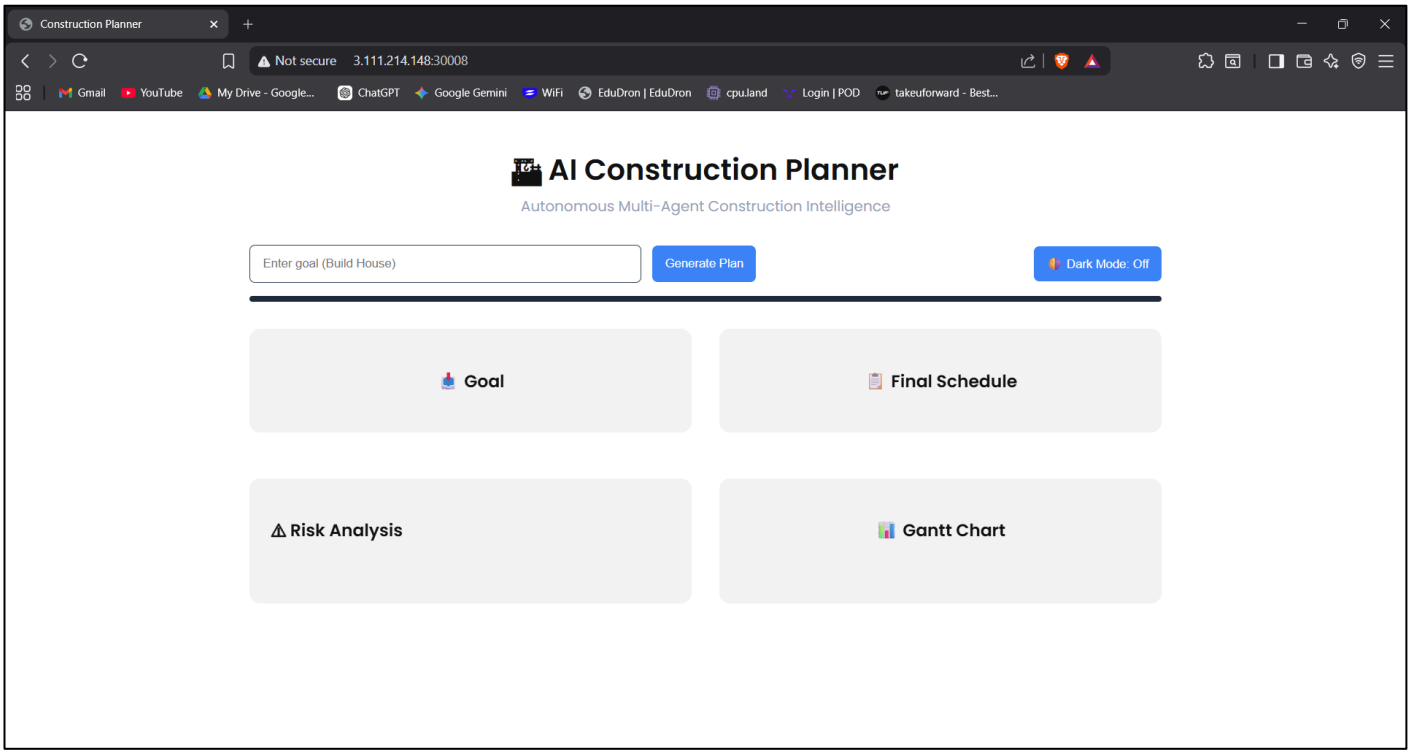


Figure 6: Application Deployed on AWS EC2 using Kubernetes

2. Reports / Models

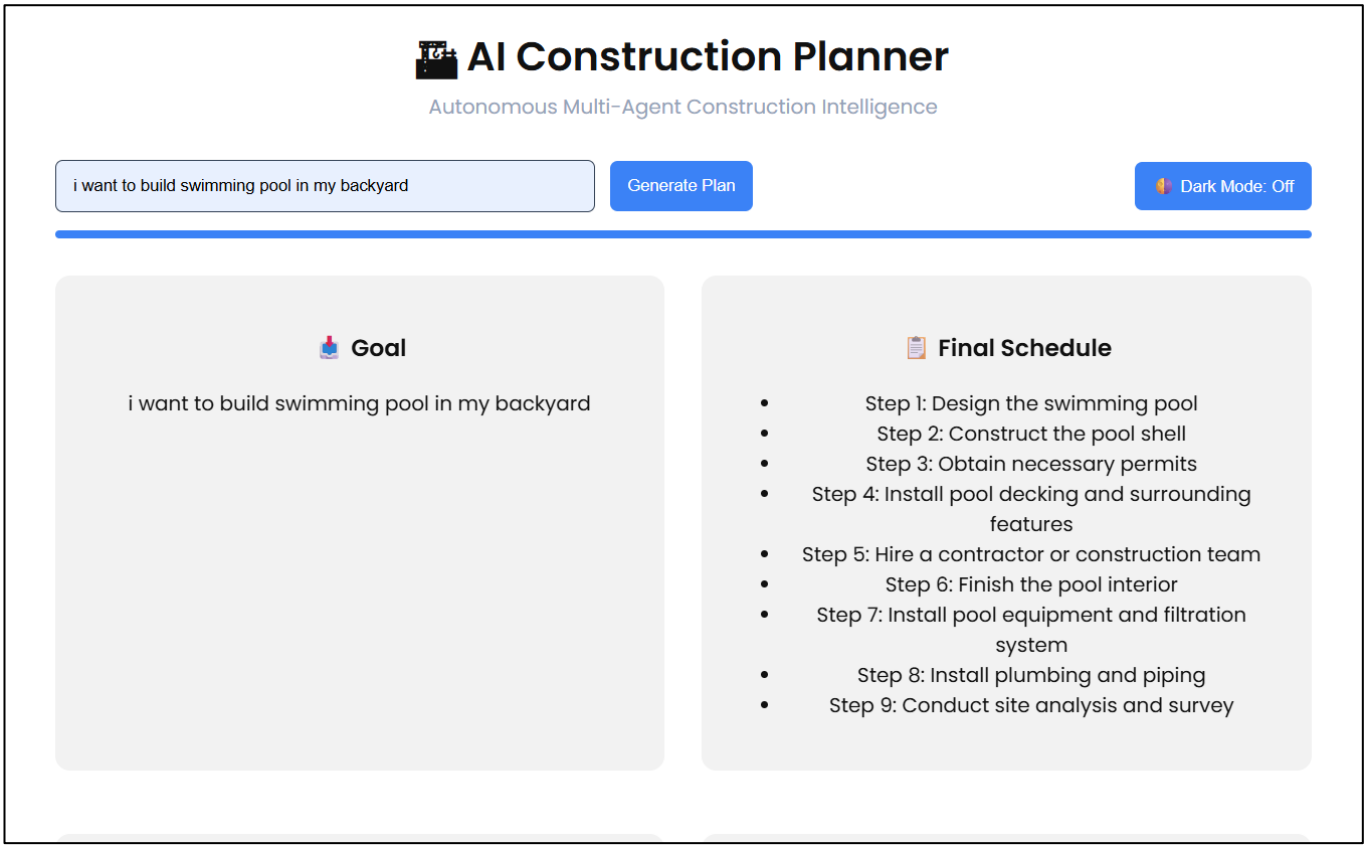


Figure 7: Generated Construction Schedule

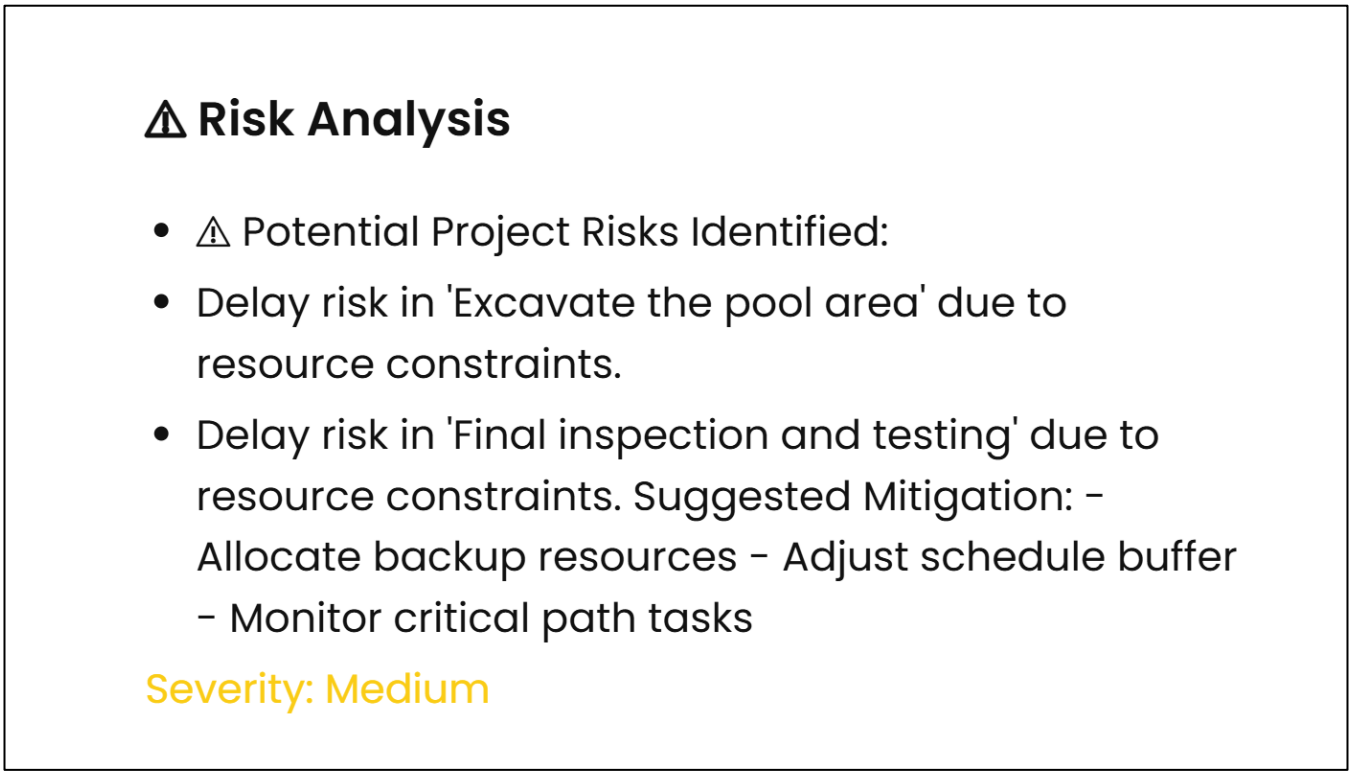


Figure 8: Risk & Severity Analysis Output

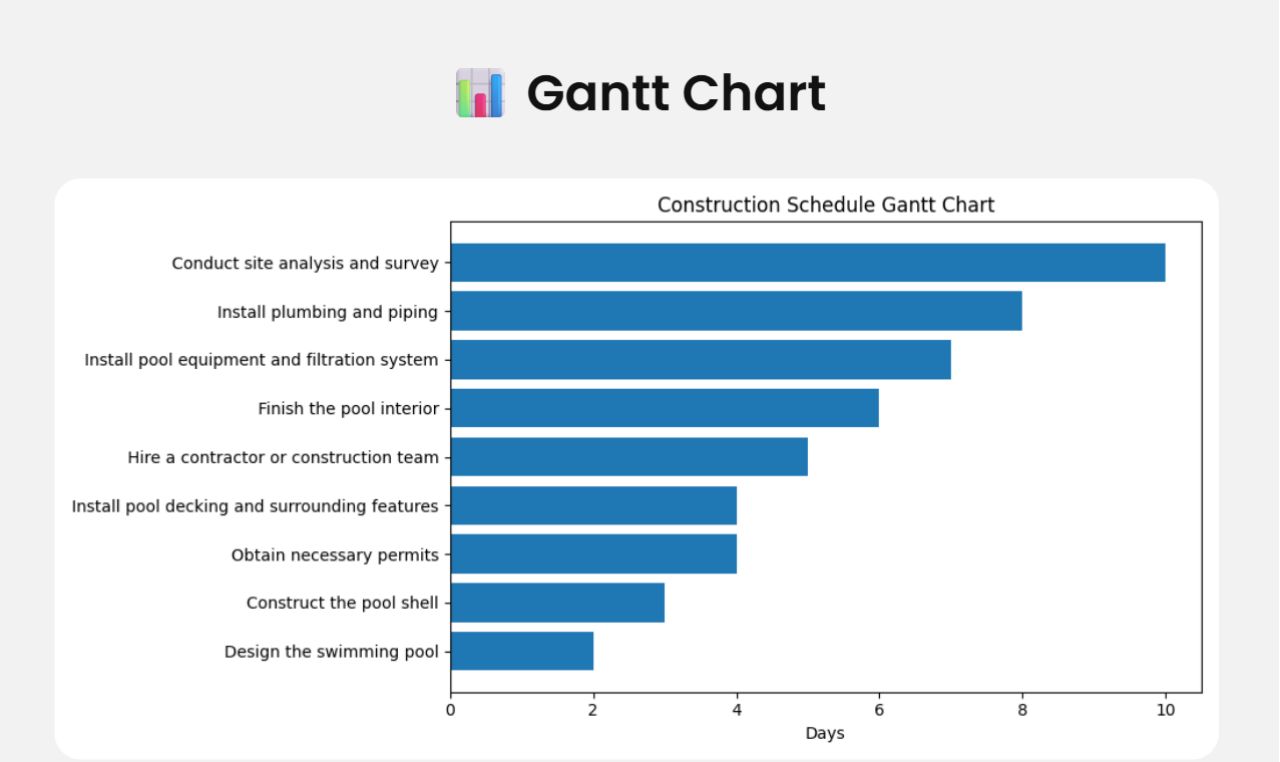


Figure 9: Gantt Chart Visualization

3. Key Outcomes

- Successfully containerized the application using Docker
- Achieved consistent environment across systems
- Deployed application using Kubernetes cluster
- Automated build process using GitHub Actions
- Hosted application on AWS EC2
- Implemented scalable microservices architecture

Conclusion

The Dockerized Construction Planning Service demonstrates a complete DevOps lifecycle by integrating containerization, orchestration, cloud deployment, and automation. The project successfully achieves portability, scalability, and efficiency using Docker and Kubernetes. The implementation of CI/CD ensures faster and reliable development workflows. The project provides practical exposure to real-world DevOps tools and cloud infrastructure.

Future Scope & Enhancements

- Integration of a database such as MongoDB or PostgreSQL
- Implementation of Continuous Deployment (CD) pipeline
- Deployment using AWS EKS for production-grade scalability
- Implementation of load balancing using AWS ELB
- Addition of authentication and authorization mechanisms
- Integration of monitoring tools such as Prometheus and Grafana
- Implementation of auto-scaling using Kubernetes HPA